

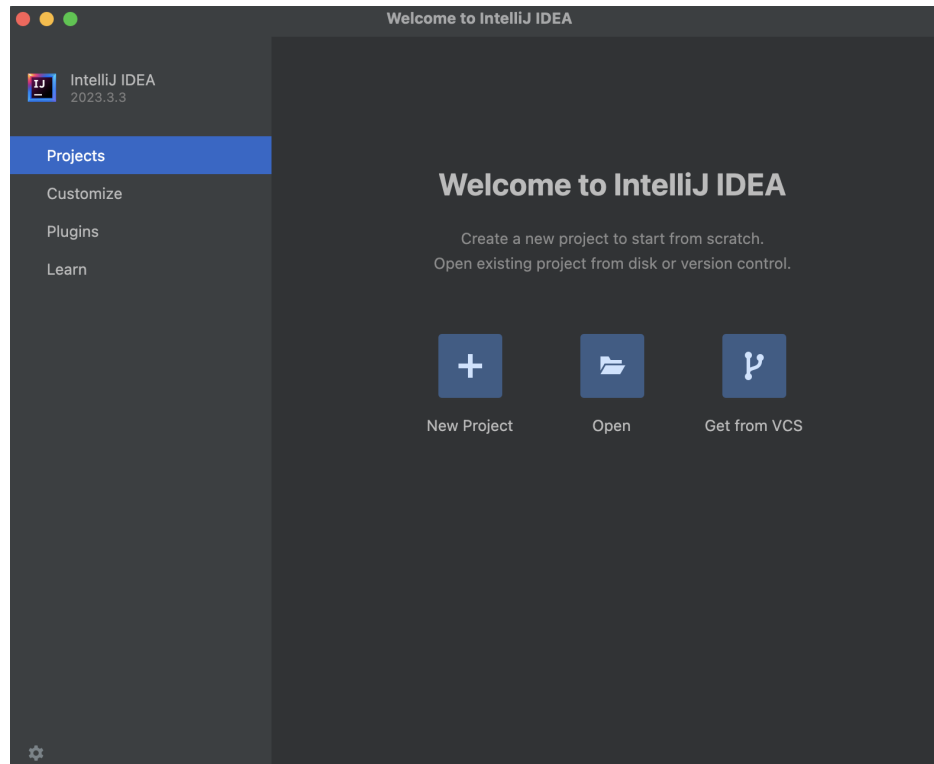
DEV2 – JAVL – Laboratoire Java**TD 01 – Intro Orienté Objet***Introduction à l'IDE IntelliJ, au langage Java et à l'orienté objet***Résumé**

Dans ce TD vous allez découvrir l'environnement intégré IntelliJ, et créer votre premier programme orienté objet en utilisant le langage Java.

1 L'environnement de développement intégré IntelliJ**Tutoriel 1 Premier programme Java avec IntelliJ**

Vous allez être guidé pas à pas pour la création de votre tout premier programme Java.

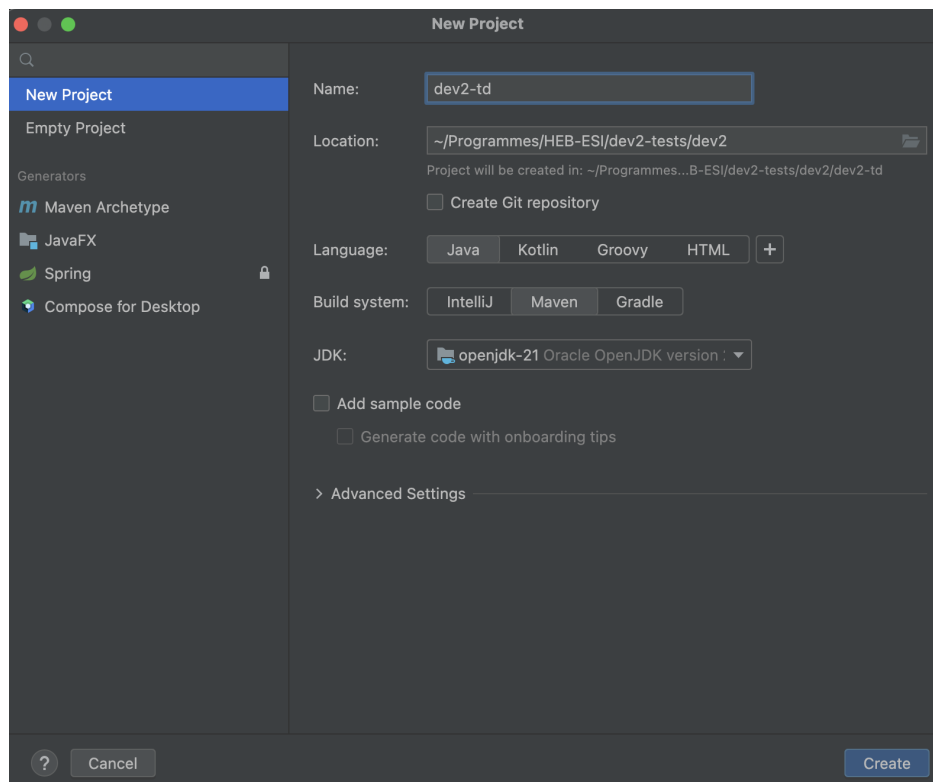
✍ Ouvrez IntelliJ : l'icône de l'application se trouve sur votre Bureau.



L'interface qui s'ouvre vous permet de parcourir les **projets** Java dont IntelliJ connaît l'existence, ainsi que d'accéder à des paramètres de base tels que l'apparence de l'environnement.

✎ Créez un nouveau projet :

Pour cela, commencez par cliquer sur le bouton "New Projet" en haut à droite de la fenêtre. Une nouvelle fenêtre s'ouvre, dans laquelle vous devrez renseigner les informations de base du projet.



Sur la gauche se trouvent différentes options de projets pré-remplis ; contentez-vous de "New Project". Sur la droite se trouvent les informations dont IntelliJ a besoin :

- ▷ Le *nom* du projet
- ▷ L'*emplacement* du projet dans l'architecture du fichiers
- ▷ Le *langage* de programmation utilisé dans le projet (IntelliJ gère plusieurs langages)
- ▷ Le *gestionnaire de production* qui permettra de configurer le projet
- ▷ Le kit de développement Java *JDK* qui contient les outils permettant de compiler et exécuter un programme en Java

Dans le cadre du cours, nous utiliserons la configuration suivante :

- ▷ Nommez le projet **dev2-td**
- ▷ L'emplacement est laissé à votre choix. *Si vous utilisez un ordinateur de l'école*, il est recommandé d'utiliser un répertoire sur le disque Z :
- ▷ Le langage sera **Java**
- ▷ Le gestionnaire de production sera **Maven**¹
- ▷ Le JDK recommandé est **21 Oracle OpenJDK**
- ▷ **Décochez** la case "Add sample code" ; cela créera du code inutile qui alourdira notre projet.

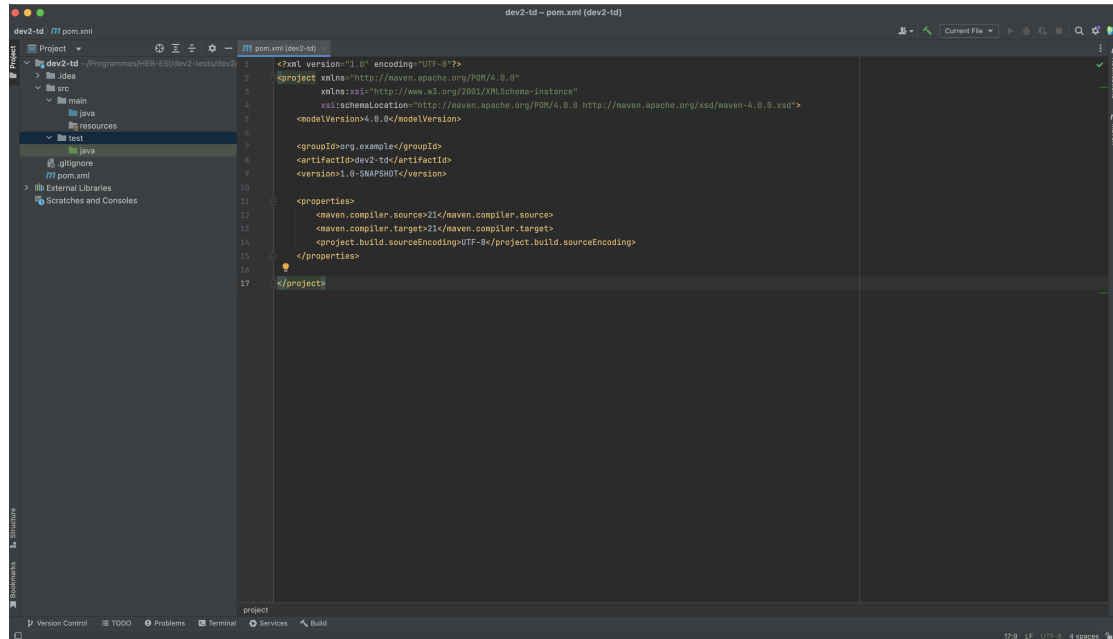
1. Pensez à **toujours** utiliser Maven quand vous créez un projet pour DEV2. Nous utiliserons cette spécificité par la suite. Si vous n'êtes pas sûr : vérifiez qu'un fichier `pom.xml` existe à la racine de votre projet. Si oui, c'est normalement correct.

Une fois les informations correctement entrées, sélectionnez "Create". Votre projet est créé !

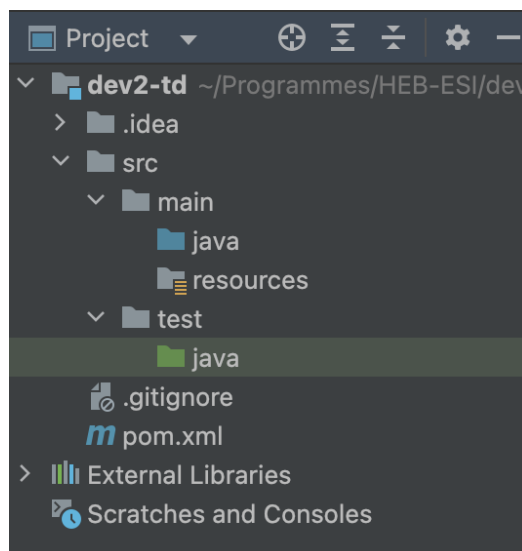
Tutoriel 2

Prise en main de l'interface

Une nouvelle fenêtre s'est ouverte, qui affiche le contenu de votre projet.



- ✍ Sur la gauche se trouve l'*explorateur de projet*. Celui-ci vous montre l'architecture des fichiers de votre projet. Notons tout de suite :



- ▷ Le dossier `.idea` contient des fichiers de configuration utilisés par IntelliJ ; n'y touchez pas.
- ▷ Le fichier `pom.xml` contient la configuration de Maven. Pour l'instant, il renseigne l'identification du projet et la version du langage Java utilisée.
- ▷ Le dossier `src` dans lequel se trouveront les fichiers du code de votre programme. A l'intérieur, deux sous-dossiers sont déjà présents :
 - ▷ `main` contient le code principal du programme.

- ▷ **test** contient les fichiers de tests, dont nous reparlerons dans un futur TD.

2 Premier programme orienté objet

Maintenant que le projet est créé, vous allez pouvoir écrire votre premier programme *orienté objet*.

Orienté objet En **programmation orientée objet** (OO), un programme est composé d'*objets*, qui représentent des entités stockées dans la mémoire de l'ordinateur. Ces objets interagissent via l'envoi de messages représentés par des appels de **méthodes**, des fonctions attachées à un objet. Chaque objet est dès lors responsable de *connaître son propre comportement*.

Tutoriel 3

Définir un chien qui aboie

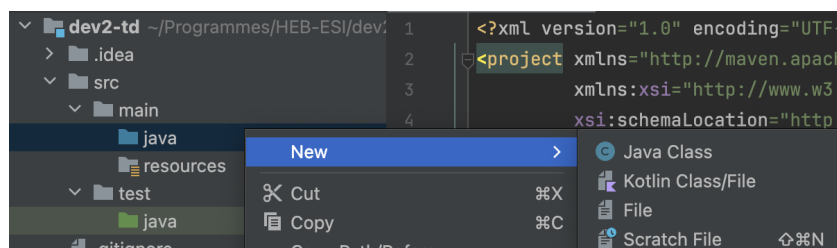
Dans ce programme, vous allez représenter un chien (*virtuel*, bien entendu).

- ✍ La première étape consistera donc à décrire ce qu'est un chien, dans le contexte de votre programme. Pour cela, vous allez créer une première *classe*.

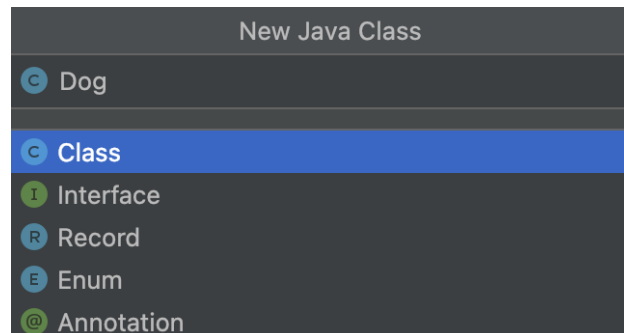
Classes Afin de définir ce qu'un objet peut faire dans un programme OO, on définit des **classes**. Chaque objet appartient à une classe, qui définit (notamment) les méthodes que cet objet possède.

Dans l'explorateur de projet d'IntelliJ, ouvrez (si ce n'est pas déjà fait) le dossier **src**, puis son sous-dossier **main**. Là, un dossier **java** vous attend. Il est marqué en **bleu** : c'est le symbole qu'IntelliJ utilise pour vous montrer que ce dossier est celui où vous stockerez *tout le code Java* de votre programme.

Effectuez un clic droit sur ce dossier : un menu apparaît. La première option est "New", qui vous permet de créer un nouveau fichier. Choisissez "Java Class".



- ✍ Une boîte apparaît. Vous pouvez y *nommer* votre classe, et choisir l'un de plusieurs archétypes. Laissez l'option "Class" et nommez votre classe `Dog`, puis appuyez sur la touche Enter au clavier.



- ✍ La classe `Dog` s'ajoute dans l'explorateur de projet, et dans la fenêtre principale, le fichier contenant le code de la classe apparaît, même s'il est pour l'instant assez vide.

```
public class Dog {  
}
```

Anatomie d'une classe Java En Java, une classe est définie avec les notations suivantes :

- ▷ Le mot-clé `class` indique que nous définissons une nouvelle classe, il est suivi du **nom** de la classe
- ▷ Après le nom de la classe, des **accolades** `{}` vont délimiter le *contenu* de la classe : tout ce qui se trouve entre ces accolades fait partie du code de la classe, ce qui est en-dehors des accolades n'en fait pas partie.
- ▷ Le mot-clé `public`, placé tout devant, est la **visibilité** de la classe ; nous y reviendrons plus tard.

- ✍ Nous pouvons maintenant définir ce qu'un chien est capable de faire : en particulier, nous allons permettre aux chiens d'aboyer. Pour cela, nous allons créer une première *méthode* dans la classe `Dog`.

Méthode Une **méthode** est un type de fonction utilisé en programmation orientée objet. Une méthode est toujours appelée sur un objet spécifique, et désigne le comportement de *cet* objet particulier.

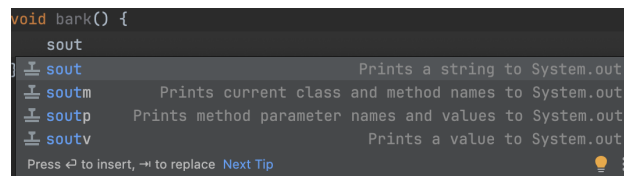
A l'intérieur de la classe `Dog` (donc entre les accolades), créez la méthode comme ceci :

```
void bark() {  
  
}
```

Anatomie d'une méthode Java En Java, une méthode doit être définie *au sein d'une classe*, avec les notations suivantes :

- ▷ Un mot-clé qui définit le **type de retour** de la méthode, c'est-à-dire le type du résultat renvoyé par la méthode. Puisqu'ici nous n'avons pas de résultat, le mot-clé `void` ("vide") est utilisé.
- ▷ Le nom de la méthode, suivi de parenthèses. Si la méthode prenait des paramètres, ils seraient placés entre ces parenthèses.
- ▷ Entre accolades, le **code** de la méthode.

Pour l'instant, cette méthode est vide. Placez-vous à l'intérieur de la méthode, et écrivez `sout`. Il s'agit d'un raccourci d'IntelliJ pour appeler `System.out.println`, la fonction Java qui permet d'écrire du texte en console, similaire au `print` de Python. Un pop-up apparaît qui vous propose quel raccourci vous souhaitez employer :



Appuyez sur la touche Enter. Vous pouvez à présent écrire le texte que vous souhaitez afficher lorsqu'un chien aboie. Par exemple :

```
void bark() {  
    System.out.println("Wouf !");  
}
```

Tout comme en Python, une chaîne de caractère en Java est entourée de guillemets ". Notez aussi que l'instruction est terminée par un **point-virgule** ;. En Java, il faudra placer un point-virgule après chaque instruction pour s'assurer qu'elles sont séparées correctement.

Votre classe doit donc ressembler à ceci :

```
public class Dog {  
    no usages  
    void bark() {  
        System.out.println("Wouf !");  
    }  
}
```

IntelliJ annote votre code IntelliJ ajoute de nombreuses informations au sein du code, et il est utile d'apprendre à les lire !

Par exemple, ci-dessus, on peut voir le message "no usages". Ceci indique que la méthode `bark` n'est pour l'instant jamais utilisée dans notre programme. De même, le nom de la classe `Dog` et de la méthode `bark` sont grisés, ce qui indique aussi qu'ils ne sont pas encore utilisés. N'ignorez pas ces outils, qui vous permettront de déboguer votre code beaucoup plus facilement !

Nous avons défini ce qu'est un chien : un objet capable d'aboyer. Et maintenant ? Ce n'est pas encore un programme ! Où sont les instructions ?

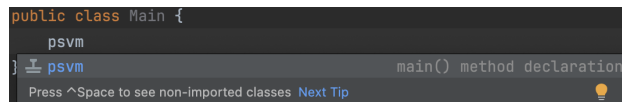
En Java, il est impossible d'écrire du code en-dehors d'une classe. Le point de départ de notre programme sera en fait lui aussi défini dans une *méthode* ! On nommera celle-ci la **méthode principale** du programme (*main* en anglais).

- ✍ Créez une classe `Main`.

Par convention, en Java, on placera la méthode principale dans une classe à part. C'est à cela que sert cette deuxième classe.

- ✍ Placez-vous à l'intérieur de la classe `Main` (entre les accolades ; n'hésitez pas à faire un retour à la ligne pour vous donner de la place).

IntelliJ propose un raccourci pour créer la fonction principale de votre programme : **psvm**. Entrez ces quatre lettres dans votre code : un menu apparaît, qui propose la déclaration de la méthode principale.



Appuyez sur Enter pour confirmer. Le code apparaît pour vous !

```
public class Main {  
    public static void main(String[] args) {  
  
    }  
}
```

La méthode *main* en Java En Java, la méthode principale du programme possède une structure bien définie :

- ▷ Elle s'appelle `main`
- ▷ Elle reçoit les mots-clés `public static` dont nous reparlerons plus tard
- ▷ Son type de retour est `void` (pas de résultat)
- ▷ Elle prend un argument de type `String[]` (un tableau de chaînes de caractère) qui représente des paramètres passés au programme par exemple dans un terminal. Nous ne les utiliserons pas dans le cadre du cours, mais il faut malgré tout dire à Java qu'ils sont présents.

Notez que le raccourci PSVM est un acronyme pour **P**ublic **S**tatic **V**oid **M**ain, puisque ces mots-clés sont toujours présents lorsqu'on définit la méthode principale.

- ✍ Il est maintenant temps de remplir cette méthode.

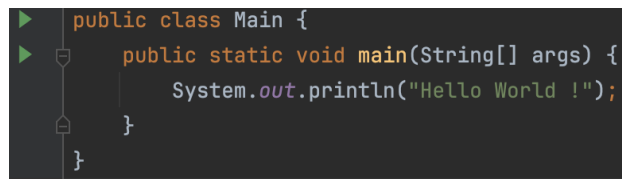
Pour faire un premier test, nous allons effectuer un *print* directement dans la méthode principale. Entrez donc une instruction, par exemple

```
System.out.println("Hello World !");
```

N'oubliez pas le raccourci `sout` !

✎ Enfin, lançons ce programme de test.

Vous aurez peut-être remarqué que, dès que vous avez ajouté la méthode `main` à votre classe, des boutons "play" sont apparus sur le côté de votre code :



```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World !");  
    }  
}
```

Une autre option pour lancer le code est d'utiliser les boutons en haut à droite de la fenêtre d'IntelliJ :



Lorsque vous sélectionnez l'un d'eux, l'option "Run Main.main()" apparaît : sélectionnez-la.

En bas de l'écran, le panneau "Run" s'ouvre. Il s'agit de la console utilisée lors de l'exécution de votre programme. Si tout s'est bien passé, elle ressemble à ceci :



La première ligne reprend les informations concernant le lancement du programme, et la dernière ligne est un message qui vous donne le code de sortie (0 signifie qu'aucun problème n'a eu lieu). Le reste reprend les affichages en console, et donc ici notre "Hello World!"

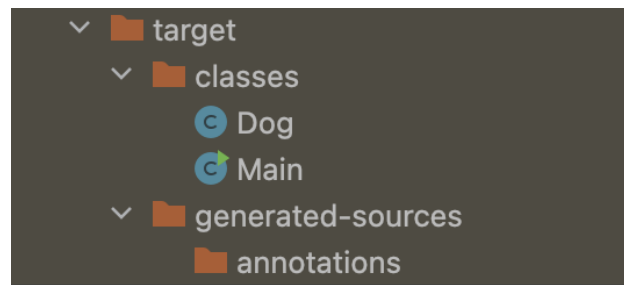
✎ Petite pause...

Maintenant que vous avez lancé le programme une première fois, prenons le temps de voir ce qui a changé dans le projet.

Java est un langage *compilé*² : vos fichiers de codes Java ont donc été compilés en des fichiers `.class` avant de pouvoir être exécutés.

2. Il est en fait compilé, puis interprété ; nous vous renvoyons au cours théorique pour plus de détails.

Ces fichiers sont placés dans le dossier **target**, qui vient d'apparaître dans votre explorateur de projet :



Si quelque chose ne va pas lorsque vous lancez votre programme, il peut être utile de garder en tête que le programme qui est réellement exécuté sont les fichiers `.class` présents dans ce répertoire.

✍ Revenons à notre chien.

Maintenant que vous savez comment lancer un programme en Java, il est temps d'y créer un *objet*. Pour cela, nous allons bien sûr employer la classe `Dog` que nous venons de créer.

Dans la méthode `main`, écrivez le code suivant :

```
Dog médor = new Dog();
```

Cette instruction consiste en deux opérations :

- ▷ La *création* (aussi appelée *instantiation*) d'un objet avec `new Dog()`. Le mot-clé `new` signale qu'on crée un objet, suivi du nom de la classe à laquelle cet objet appartiendra. Les parenthèses permettront, le cas échéant, de donner des paramètres : nous n'en avons pas ici, donc elles restent vides (mais sont *obligatoires*!)
- ▷ L'*assignation* de cet objet à une variable. Un objet, comme n'importe quelle autre valeur, peut être stocké dans une variable, qui nous permettra de faire référence à cet objet dans le reste de notre programme.

Java : un langage typé Java est un langage **fortement typé** : il faudra toujours préciser le type des valeurs avec lesquelles on travaille. Vous l'avez déjà vu lors de la définition d'une méthode : on doit préciser le type de sa valeur de retour.

De même, lorsqu'on utilise une variable pour la première fois en Java, il faudra *déclarer* celle-ci en précisant son type :

```
Dog médor;
```

Une fois le type d'une variable déclarée, ce type ne peut plus changer. Une variable en Java aura donc toujours le même type, tant qu'elle existe.

Nous avons maintenant une variable qui fait référence à un objet de la classe chien. Il est maintenant possible de dire à ce chien quoi faire en appelant une de ses méthodes. Ajoutez l'instruction suivante :

```
médor.bark();
```

Lancez ensuite le programme, qui affiche maintenant :

Wouf !

Process finished with exit code 0

Vous y voilà ! Votre premier programme orienté objet est terminé : vous avez créé un objet, et vous avez défini votre méthode principale où vous lui avez dit quoi faire.

3 Bien maintenir son code

Plus vous avancez dans votre carrière de développeur, plus vos programmes vont gagner en complexité. Il est important de prendre des bonnes habitudes à ce sujet, et l'une d'entre elles consiste à **documenter** votre code, c'est-à-dire rédiger des textes informatifs pour qu'une personne qui lise votre code puisse comprendre d'un regard le rôle de chaque élément de votre programme sans devoir lire le code en lui-même.

En Java, le standard utilisé est la *Javadoc* : devant chaque élément du code (définition de classe, de méthode, ...), on placera un commentaire structuré selon les règles définies par la Javadoc afin d'expliquer de quoi il s'agit. Une fois les commentaires placés, la Javadoc peut être utilisée pour générer une documentation du programme sous la forme de pages Web.

Les commentaires Javadoc en Java commencent toujours par `/**` et se terminent par `*/`. IntelliJ possède en outre des outils pour vous aider à écrire la Javadoc dans votre code.

- ✍ Par exemple, placez-vous au-dessus de l'en-tête de la méthode `bark` de la classe `Dog`, et écrivez `/**...`

```
public class Dog {  
    /**  
    void bark() {  
        System.out.println("Wouf !");  
    }  
}
```

Lorsque vous appuyez sur Enter, un squelette de commentaire Javadoc est créé pour vous. Ici, il est vide ; cependant, si l'en-tête de la fonction contient des informations (type de retour, paramètres...), ces informations seront incluses dans le commentaire généré.

- ✍ Il ne vous reste plus qu'à rédiger une description brève !

```
/**  
 * Tell the dog to bark  
 */  
1 usage  
void bark() {
```

- ✍ De même, si vous vous placez avant la définition de la classe, vous pouvez créer un commentaire Javadoc où vous décrirez ce que représente un objet de cette classe.

```
/**
 * Represents a dog inside the program
 * @author tnicodeme
 */
2 usages
public class Dog {
```

La ligne qui commence par `@author` permet de renseigner le créateur de la classe ; il existe de nombreuses annotations de ce type, qui permettront de structurer la documentation.

- ✍ Une fois que vous avez documenté votre code, allez dans le menu "Tools -> Generate Javadoc..." (en haut de la fenêtre d'IntelliJ).

Une boîte de dialogue s'ouvre et vous demande de configurer la Javadoc ; il vous faudra juste préciser où placer la documentation générée. Choisissez un dossier : créez par exemple un sous-dossier dans `target`. Dans le menu "Visibility level", choisissez aussi "package" pour le moment, puisque nous n'avons pas encore réglé la visibilité de nos éléments.

Quand tout est réglé, cliquez sur "Generate". Les fichiers sont créés, et la documentation s'ouvre automatiquement dans votre navigateur Web par défaut :

PACKAGE

CLASS

TREE

INDEX

HELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES

Unnamed Package

Classes

Class

Dog

Main

Description

Represents a dog inside the program

La page d'accueil reprend les différentes classes de votre projet : si vous choisissez la classe `Dog`, vous verrez alors les informations que vous avez placées dans vos commentaires, repris de façon structurée.

Class Dog

java.lang.Object¹²

Dog

public class Dog

extends Object¹²

Represents a dog inside the program

Constructor Summary

Constructors

Constructor	Description
Dog()	

Method Summary

All MethodsInstance MethodsConcrete Methods

Modifier and Type	Method	Description
(package private) void	bark()	Tell the dog to bark

Prenez le temps de commenter votre code ! Dans tout travail de programmation, la **maintenance** du code est nettement plus coûteuse que son *écriture*. Il est donc essentiel de rendre le code aussi lisible que possible, et la documentation est un outil pratique dans ce but. Prenez donc le réflexe de toujours commenter votre code Java *en utilisant des commentaires Javadoc* !

4 Le chat

Nous voulons maintenant créer une nouvelle classe **Cat**. Les chats sont identiques à nos amis canidés, à ceci près qu'un chat n'aboie pas, mais il miaule. Il faudra donc supprimer la méthode **bark** et écrire une méthode équivalente **meow**.

Exercice 1

La classe Cat

Créez une classe **Cat** qui est en tout point la même que la classe **Dog** à l'exception que la méthode **bark** est remplacée par une méthode **meow**. N'oubliez pas d'adapter les affichages (remplacer "wouf" par "miaou").

Répétition du code ?

Cette façon de faire n'est pas la plus pratique, car elle vous demande de copier du code. Pour surmonter ce désagrément (et d'autres qui en découlent), nous allons introduire une nouvelle classe et la notion d'héritage dans un TD ultérieur.